

PKM Veille Technologique

Documentation technique complète

Corentin Godon — Mastère 1 Expert IT, Développement & BDD
Épreuve E1 — Veille Technologique

Déployé sur Fly.io (Paris, cdg) · Bot Discord 24h/24

1. Présentation du projet

Ce projet est un système de **PKM (Personal Knowledge Management)** automatisé, conçu pour l'épreuve E1 de veille technologique. Il collecte, résume et met à disposition des articles techniques en lien avec le développement web (frontend, backend, sécurité), via un bot Discord accessible 24h/24.

Le pipeline complet : capturer l'information brute (RSS) → la scorer par IA → la valider manuellement → la stocker → la distribuer dans les bons salons thématiques → la rendre interrogeable en langage naturel via la commande /ask.

2. Architecture globale

Pipeline de traitement de l'information :

Sources RSS (13 sources) + soumission manuelle /submit
↓ Collecte TypeScript (toutes les 2h, 3 articles max/source, délai 1s)
↓ Mistral AI – Scoring de pertinence 1-5
↓ Score ≥ 4 → Mistral AI – Résumé en français (3-4 phrases)
↓ #veille-validation – Boutons ■ Valider / ■ Rejeter + ■ Score affiché
↓ Si validé → SQLite (volume persistant Fly.io)
■■■ #veille-frontend → Frontend & Frameworks JS [■ En savoir plus]
■■■ #veille-backend → Backend, API & Outillage [■ En savoir plus]
■■■ #veille-securite → Sécurité Web [■ En savoir plus]
↓ #veille-digest – Digest hebdomadaire (lundi 8h Paris)

Tout le pipeline tourne en continu sur **Fly.io région Paris (cdg)**, avec un volume persistant SQLite. Les articles en attente de validation et les données des boutons "En savoir plus" sont persistés en base pour survivre aux redémarrages du bot.

3. Organisation des salons Discord

Salon	Rôle
#veille-commandes	Toutes les commandes slash
#veille-validation	Articles à valider (boutons  + score )
#veille-frontend	Articles validés — Frontend (bouton  En savoir plus)
#veille-backend	Articles validés — Backend (bouton  En savoir plus)
#veille-securite	Articles validés — Sécurité (bouton  En savoir plus)
#veille-digest	Résumé hebdomadaire automatique chaque lundi 8h

4. Stack technique

Composant	Technologie	Justification
Runtime	Node.js 20 + TypeScript	Typage fort, écosystème riche
Collecte RSS	rss-parser	Léger, sans dépendance lourde
IA — Scoring	Mistral AI (mistral-small-latest)	Souveraineté française 
IA — Résumés	Mistral AI (mistral-small-latest)	Souveraineté française 
IA — RAG /ask	Mistral AI + Tavily Search	Recherche locale + fallback web
IA — En savoir plus	Mistral AI (retry x3)	Analyse approfondie à la demande
IA — Digest	Mistral AI	Synthèse hebdomadaire automatique
Stockage	SQLite via better-sqlite3	Zéro configuration, persistant
Bot	discord.js (commandes slash)	API officielle Discord, gratuit
Hébergement	Fly.io (Paris cdg)	Docker natif, volume persistant
Versionning	GitHub (repo privé)	CI/CD, traçabilité

5. Sources de collecte (13 sources)

Axe	Sources	Salon cible
Frontend & Frameworks JS	Dev.to, Smashing Magazine, This Week in React, web.dev, Alsacréations	#veille-frontend
Backend, API & Outillage	Node Weekly, JavaScript Weekly, TypeScript Blog, Bun Blog, GitHub Changelog	#veille-backend
Sécurité Web	The Hacker News, CERT-FR (ANSSI)  , Zero Day Initiative	#veille-securite

Collecte toutes les **2 heures**, 3 articles max par source, délai 1s anti-rate-limit. Les sources sont dynamiquement gérables via /rss-add et /rss-remove (stockées en SQLite). Si aucune source n'est en base, les sources par défaut s'appliquent.

6. Commandes disponibles (10 commandes)

Commande	Description
/summary	Affiche les 5 derniers articles validés (titre, source, résumé, lien)
/ask [question]	RAG : recherche locale SQLite + fallback Tavily + synthèse Mistral
/export	Génère un fichier Markdown complet et l'envoie en pièce jointe
/analyze	Analyse IA : top 3 tendances, sources actives, lacunes, recommandations
/submit [url]	Soumet un article → résumé Mistral → envoi dans #veille-validation
/rss-list	Liste les sources RSS actives avec leur ID
/rss-add [nom] [url]	Ajoute une source RSS (persistée en SQLite)
/rss-remove [id]	Supprime une source RSS par son ID
/infos	Statut du système : collecte, articles en base, infrastructure
/help	Liste complète des commandes (message éphémère)

7. Fonctionnement détaillé

7.1 — Collecte, scoring et résumé (toutes les 2h)

1. Fetch de chaque flux RSS séquentiellement (délai 1s), 3 articles max/source.
2. Filtrage des articles déjà vus via la table *seen_urls*.
3. Mistral AI attribue un score de pertinence 1 à 5 — l'article est marqué vu immédiatement.
4. Seuls les articles avec un score ≥ 4 sont résumés en 3-4 phrases en français.
5. L'article est envoyé dans #veille-validation avec le score (■ Score : X/5) et les boutons ■/■.
6. Les articles en attente sont persistés en SQLite (table *pending_validations*) — les boutons restent fonctionnels après un redémarrage du bot.

7.2 — Validation et routing thématique

■ **Valider** → insertion en SQLite + post automatique dans le bon salon thématique avec le bouton ■ En savoir plus. ■ **Rejeter** → article ignoré définitivement.

Source	Salon cible
Dev.to, Smashing Magazine, This Week in React, web.dev, Alsacréativité	#veille-frontend
Node Weekly, JavaScript Weekly, TypeScript Blog, Bun Blog, GitHub Changelog	#veille-backend
The Hacker News, CERT-FR (ANSSI), Zero Day Initiative	#veille-securite
Articles soumis via /submit	#veille-backend (par défaut)

7.3 — Bouton ■ En savoir plus

Mistral génère une analyse approfondie (contexte, points clés, pistes pour aller plus loin).
La réponse est visible uniquement par toi (message éphémère Discord).
Les données sont persistées en SQLite (table *more_info_cache*) pour survivre aux redémarrages.
Retry automatique x3 avec délai 30s en cas de rate limit Mistral (erreur 429).

7.4 — Digest hebdomadaire automatique

Chaque **lundi à 8h heure de Paris**, le bot poste dans #veille-digest : top tendances de la semaine, articles incontournables (2-3 max), et ce qu'il faut retenir. Le scheduler vérifie toutes les heures le jour et l'heure en timezone Europe/Paris.

7.5 — Commande /ask — Pipeline RAG

1. Recherche locale : requête LIKE sur titre, résumé et contenu dans SQLite.
2. Fallback web : si moins de 2 résultats locaux → appel Tavily Search API (3 résultats).
3. Synthèse Mistral : contexte local + web → réponse structurée en français.

8. Base de données SQLite — 5 tables

Table	Rôle
articles	Articles validés (titre, url, source, résumé, contenu, date)
seen_urls	URLs déjà traitées — évite les doublons entre collectes
rss_sources	Sources RSS gérées dynamiquement via les commandes Discord
pending_validations	Articles en attente de validation — persistés pour survivre aux redémarrages
more_info_cache	Données des boutons En savoir plus — persistés pour survivre aux redémarrages

9. Variables d'environnement

Variable	Description
MISTRAL_API_KEY	Clé API Mistral AI (mistral.ai)
DISCORD_BOT_TOKEN	Token du bot Discord
DISCORD_CLIENT_ID	Application ID Discord (commandes slash)
DISCORD_CHANNEL_ID	Salon principal (fallback)
DISCORD_CHANNEL_VALIDATION	ID du salon #veille-validation
DISCORD_CHANNEL_FRONTEND	ID du salon #veille-frontend
DISCORD_CHANNEL_BACKEND	ID du salon #veille-backend
DISCORD_CHANNEL_SECURITE	ID du salon #veille-securite
DISCORD_CHANNEL_DIGEST	ID du salon #veille-digest
TAVILY_API_KEY	Clé API Tavily Search (fallback web pour /ask)
DB_PATH	Chemin SQLite (monté sur /data sur Fly.io)

10. Déploiement Fly.io

Infrastructure :

- Hébergeur : Fly.io (région cdg — Paris Charles de Gaulle)
- VM : 1 machine, shared-cpu-1x, 512 MB RAM
- Stockage : Volume persistant *veille_data* (1 GB, chiffré)

- Image Docker : Node.js 20 Alpine (~193 MB)

Commandes de déploiement :

```
flyctl deploy --app veille-tech-corentin
flyctl logs --app veille-tech-corentin
flyctl secrets set CLE=valeur --app veille-tech-corentin
```

11. Choix de souveraineté numérique

Composant	Solution	Origine	Alternative écartée
IA résumés + scoring	Mistral AI	■■ France	GPT-4 (USA)
IA analyse + digest	Mistral AI	■■ France	Claude API (USA)
Hébergement	Fly.io Paris cdg	■■ France	AWS/GCP (USA)
Sources FR	Alsacréations, CERT-FR (ANSSI)	■■ France	—
Base de données	SQLite (open source)	Open Source	MongoDB Atlas (USA)
Recherche web	Tavily	■■ USA	Brave (pas de free tier)
Versionning	GitHub (privé)	■■ USA	GitLab FR possible

12. Compétences E1 couvertes

Compétence	Sections couvertes
C1 — Organiser un système de veille	Sources §5, Architecture §2, Salons §3, Gestion RSS §7
C2 — Analyser les informations	Scoring §7.1, Résumés §7.1, RAG /ask §7.5, /analyze, En savoir plus §7.3
C3 — Expérimenter et mobiliser	Déploiement §10, /submit, Digest §7.4, Workflow Claude §13

Ce projet couvre l'intégralité du référentiel E1 en mobilisant une stack moderne, souveraine et entièrement automatisée, tout en conservant le contrôle humain via la validation manuelle dans Discord.